

Architecting a High-Probability Algorithmic Options Buying System for Nifty 50: Infrastructure, Strategy, and Execution

The development of a fully automated algorithmic trading system tailored specifically for the Indian derivatives market represents a profound intersection of low-latency data engineering, advanced quantitative modeling, and autonomous software development. Designing an algorithmic architecture dedicated exclusively to buying Nifty 50 options introduces a unique, highly asymmetric set of mathematical and operational challenges. The fundamental structure of the options market dictates that option buying and option selling operate on diametrically opposed probability curves. Option sellers systematically benefit from the inexorable passage of time and the contraction of implied volatility, utilizing massive margin collateral to harvest small, highly probable yields.¹ Conversely, option buying is a structurally disadvantaged, low-probability, high-reward endeavor that demands absolute precision in directional forecasting, immediate momentum detection, and rigid, emotionless exit execution.¹

Leading derivatives market experts consistently emphasize that retail traders inherently operate at a disadvantage because they utilize options buying—a mathematical instrument originally designed for portfolio hedging—as a vehicle for directional speculation.² An option buyer pays the premium upfront and fights an incessant battle against time decay; they can accurately predict the ultimate directional movement of the Nifty 50 index and still incur a 100% loss on the premium if the velocity of that movement is insufficient to offset the daily erosion of extrinsic value.¹ Therefore, a profitable Nifty 50 options buying algorithm cannot rely on slow-moving fundamentals or mean-reversion principles. It must be engineered exclusively to detect violent volatility expansions, structural opening range breakouts, and pure price velocity, executing entries and exits within milliseconds.

This report provides an exhaustive architectural blueprint and a comprehensive Product Requirements Document (PRD) for constructing a highly profitable, fully automated Nifty 50 options buying system. The proposed architecture leverages the newly introduced Google Antigravity IDE for autonomous agentic development, integrated deeply with the INDstocks market data and execution API via low-latency WebSockets. The subsequent analysis delineates the infrastructure setup, dynamic quantitative strategies inspired by institutional market dynamics, sophisticated exit mechanisms designed to mitigate theta decay, and enterprise-grade risk management protocols.

The Agentic Development Paradigm: Google

Antigravity Infrastructure

The traditional Integrated Development Environment (IDE) relies entirely on a synchronous, human-driven code generation paradigm. The deployment of a sophisticated algorithmic trading engine requires a complex, multi-threaded architecture encompassing WebSocket data streaming, asynchronous event handlers, local database management for trade logging, and external execution APIs. Google Antigravity fundamentally alters this workflow by operating as an agent-first platform powered by the Gemini 3 Pro reasoning model, effectively transforming the developer from a manual syntax programmer into a high-level system architect.³

Dual-Surface Environment and Asynchronous Orchestration

Antigravity abandons the single-window paradigm in favor of a dual-surface approach, optimizing for both synchronous coding and asynchronous agent orchestration.⁵ The primary workspace introduces an "Agent Manager" surface, acting as a "Mission Control" dashboard that allows for the deployment of multiple autonomous agents operating concurrently across different tasks.³ For an algorithmic trading system, this architecture implies that one agent can be tasked with constructing the local SQLite database for historical trade logging, while a separate, parallel agent simultaneously scripts the INDstocks WebSocket connection logic.⁶

The environment operates under configurable autonomy levels, categorizing execution into 'Off', 'Auto', and 'Turbo' modes. In 'Auto' mode, the agent decides whether to autonomously execute terminal commands based on the inferred risk of the action, pausing to request developer permission before initiating critical environmental changes.³ In 'Turbo' mode, the system auto-executes all terminal commands except those explicitly defined in a pre-configured deny list.³ This allows the algorithmic infrastructure to be scaffolded rapidly. The Editor View provides a state-of-the-art coding interface augmented with "Agent Awareness," supporting inline commands where localized quantitative logic—such as an exponential moving average (EMA) calculation or a standard deviation array—can be refactored instantly via highlighted text.³

Furthermore, the environment implements a robust "Self-Healing System" or "Self-Annealing Loop".⁶ When generating the trading bot, if an agent encounters an unhandled exception, a syntax error, or an API token rejection during dependency installation, it does not abruptly halt. Instead, it autonomously reads the terminal output, searches the documentation context, synthesizes an alternative approach, and iterates a fix without synchronous human intervention.⁹ To enforce code quality, the system supports rigorous Agent Validation Loops utilizing tools like @google/agentkit.¹⁰ By wrapping task execution in a validation step with configurable maximum retries (e.g., maxRetries: 3), the system can systematically evaluate the generated trading logic against predefined constraints.¹⁰ Appending each validation attempt to a local validation.jsonl file provides a critical audit trail, allowing the system architect to identify recurring failure patterns in the agent's logic, such as a consistent misunderstanding of the WebSocket payload structure, thereby enabling continuous refinement of the initial

orchestration prompts.¹⁰ The system continuously verifies its own work by spawning built-in browser testing tools, generating verifiable artifacts such as implementation logs and dashboard screenshots as cryptographic proof of success.⁵

Managing Context Saturation via Agent Skills and the Model Context Protocol

A critical failure point in legacy Large Language Model (LLM) coding assistants is the phenomenon of "Context Saturation"—where dumping entire algorithmic codebases, historical backtesting arrays, and extensive API documentation into the model's active memory causes latency, financial waste, and "context rot".¹¹ When an LLM is overloaded with irrelevant data, it becomes confused, leading to hallucinations in the execution logic. Antigravity mitigates this operational hazard through the implementation of localized Agent Skills and the Model Context Protocol (MCP).¹¹

For a high-frequency options trading algorithm, the system requires continuous access to INDstocks API documentation, live market schemas, and historical execution records. Rather than injecting this entirely into the context window, the system utilizes Agent Skills. Skills are specialized, agent-triggered capabilities stored locally in the `<workspace-root>/agent/skills/` directory.¹¹ Defined using simple YAML and Markdown files, these skills allow the agent to dynamically equip the specific expertise required for a localized task.¹¹ When the developer requests an update to the order execution parameters, the reasoning engine autonomously detects the intent and loads only the specific indstocks-execution skill, keeping the context window highly focused and efficient.¹²

To interact securely with the external INDstocks API without embedding hardcoded credentials in the source files, the system employs the Model Context Protocol (MCP). MCP functions as a standardized bridge, allowing the LLM to access external resources, vector stores, and execution tools in a secure, segmented manner.³ A custom MCP server built via the Python SDK can securely hold the INDstocks `YOUR_ACCESS_TOKEN` within local environment variables and expose localized interaction tools (e.g., `fetch_nifty_instruments` or `execute_market_order`) directly to the Antigravity agent.¹³ The agent can then trigger these functions through natural language requests, securely managing the API interactions behind a configured JSON structure that seamlessly passes the required Authorization: `YOUR_ACCESS_TOKEN` HTTP headers.¹⁶ This architecture ensures that sensitive brokerage credentials never touch the active LLM context or the committed codebase.¹⁸

Market Data and Execution Architecture: The INDstocks Integration

The lifeblood of any algorithmic trading system is its data pipeline and the subsequent execution latency. The theoretical profitability of an option buying algorithm is entirely dependent on its ability to react to micro-structural market changes faster than human retail

participants. INDstocks provides a robust, zero-subscription RESTful API suite offering sub-20ms execution latency and dedicated WebSockets for both market data streaming and asynchronous trade updates.¹⁹ The API enforces highly transparent pricing at a flat ₹5 per order and maintains strict rate limits that the algorithm must systematically respect to avoid temporary account suspension or order rejection.¹⁹

Instrument Resolution and Token Mapping Logistics

Before any market data can be streamed or an execution order placed, the algorithmic system must programmatically resolve the specific alphanumeric token (the Security ID) for the desired Nifty 50 options contracts.²² Unlike equity symbols which remain relatively static, options contracts are dynamic, expiring weekly and monthly, necessitating a daily automated token resolution protocol. INDstocks utilizes a centralized Instruments Master system for this purpose.

To initialize the trading day, the algorithm must execute a standard HTTP GET request to the `/market/instruments?source=fno` endpoint.²² This specific endpoint does not return a structured JSON response; rather, it returns a raw, comprehensive CSV file containing all tradable instruments specifically within the Futures and Options (FNO) segment.²² The algorithm must parse this extensive CSV file—typically utilizing a high-performance library like Pandas in Python—and filter it dynamically to locate the targeted option contracts.

The filtering logic must isolate specific rows based on exact column matches:

- **SYMBOL_NAME:** Must be matched against the base Nifty index identifier to exclude stock options.²²
- **SEGMENT:** Filtered strictly for the FNO classification.²²
- **INSTRUMENT_NAME:** Filtered for the appropriate derivative classification (e.g., OPTIDX for index options).²²
- **OPTION_TYPE:** Dynamically filtered for CE (Call European) when executing a long bullish strategy, or PE (Put European) for a short bearish strategy.²²
- **STRIKE_PRICE:** Matched exactly against the algorithmically calculated At-The-Money (ATM) or predetermined Out-of-The-Money (OTM) target strike.²²
- **EXPIRY_DATE:** Sorted chronologically to select the nearest weekly or monthly expiration, depending on the strategy's gamma requirements.²²

Once the highly specific row is isolated, the system extracts the data from the **SECURITY_ID** column.²² This unique identifier serves as the primary key for the instrument. For all subsequent WebSocket subscriptions and order executions within the Nifty derivatives segment, this numeric token must be prefixed with NFO: (e.g., NFO:51011) to conform strictly to the INDstocks instrument formatting standard.¹⁷ Using an incorrect, malformed, or outdated ID will result in a fatal `InputException: Invalid security_id` error, halting the algorithmic execution loop.²²

Dual WebSocket Architecture and Streaming Mechanics

To ensure absolute separation of concerns and maintain processing speed, INDstocks provides two distinct WebSocket channels that the algorithm must connect to and manage concurrently.¹⁷ Relying on REST API endpoints for continuous price polling or order status checking is computationally inefficient and will immediately exhaust the strict rate limits.²³

1. The Price Feed WebSocket Engine: Live market ticks, representing the raw bid/ask dynamics and last traded prices, are retrieved by maintaining a persistent TCP connection to `wss://ws-prices.indstocks.com/api/v1/ws/prices`.¹⁷ Following the initial connection handshake—which utilizes a text-only authorization string mapped to the access token—the system must send a structured JSON subscription payload.¹⁷

To optimize bandwidth and reduce processing overhead, the system should request the specific Nifty 50 options tokens in ltp (Last Traded Price) mode if only top-level breakout logic is required, or quote mode if deeper market depth and bid-ask spread analysis are necessary.¹⁷ A single WebSocket connection allows for up to 3000 instrument subscriptions simultaneously, which provides vast capacity for monitoring entire Nifty option chains concurrently.²³

2. The Order Updates WebSocket Engine: After firing an execution order, checking its status via REST introduces unacceptable latency. The system must maintain an independent connection to `wss://ws-order-updates.indstocks.com/api/v1/ws/trades`.¹⁷ By submitting a subscription payload with the action set to subscribe and the mode set to order_updates, the broker's server will proactively push JSON messages detailing every state change of the order lifecycle.¹⁷ The algorithm must parse these incoming packets to detect state transitions from O-PENDING to FILLED, immediately capturing the filled quantity and the exact average execution price to initialize the dynamic trailing stop-loss arrays.¹⁷ Both WebSockets must implement robust heartbeat monitoring; the server sends periodic ping messages to keep the connection alive, and the algorithmic client must be configured to appropriately handle or acknowledge these heartbeats to prevent connection termination.¹⁷

Order Placement Specifications and Rate Limit Enforcement

Executing a trade via the INDstocks API requires a strictly formatted, authenticated HTTP POST request directed to the /order endpoint.¹⁷ The JSON payload must specify several critical, case-sensitive parameters to pass the broker's internal validation engine.¹⁷

For a high-probability Nifty 50 option buying algorithm aiming for an intraday momentum scalping strategy, the payload must explicitly define:

- `txn_type`: Set to "BUY" for entry, and "SELL" for exit.²²
- `exchange`: Set to "NSE".²²
- `segment`: Set to "DERIVATIVE" for F&O contracts.²²
- `product`: Defined as "MARGIN" for standard derivatives, or "INTRADAY" depending on the broker's specific leverage profile and auto-square-off requirements.²²
- `order_type`: Set to "MARKET" for guaranteed immediate execution on a high-velocity

momentum breakout, or "LIMIT" with a specified limit_price to prevent excessive slippage in illiquid strikes.¹⁷

- validity: Set to "DAY" for standard orders, or "IOC" (Immediate or Cancel) for strict limit fills.²²
- security_id: The previously resolved numerical token extracted from the CSV master.¹⁷
- qty: Must be a strict mathematical multiple of the current Nifty 50 lot size (e.g., 25, 50, 75) and must fall strictly below the exchange-mandated maximum freeze quantity.¹⁷
- algo_id: The documentation strictly mandates the string value "99999" for all regular automated API orders.¹⁷

The system architecture must aggressively enforce API rate limits locally before transmitting any requests. INDstocks caps order APIs at 10 requests per second, with a maximum of 25 modifications per order.²³ Data and Quote APIs are capped at 5 requests per second and 100,000 per day, while non-trading APIs (profile, funds) are limited to 15 per second.²³ Exceeding these thresholds triggers a HTTP 429 Too Many Requests error, resulting in dropped executions and potential catastrophic exposure if a stop-loss order is rejected.²³ Therefore, the order execution module must utilize an asynchronous token bucket algorithm or a local queueing mechanism to throttle outbound requests, ensuring the 10 orders-per-second threshold is never breached even during extreme market volatility.

Quantitative Strategy Formulation: High-Probability Option Buying

The mathematical realities of the Black-Scholes options pricing model dictate that option buying is structurally penalized. Leading institutional quantitative traders and derivatives experts constantly reiterate that while option sellers rely on high-probability, low-yield mechanics fueled by intrinsic mathematical decay and large margin deployment, option buyers must rely on low-probability, high-yield anomalies driven exclusively by explosive directional momentum and rapid delta expansion.¹ An option buyer's underlying view of the market can be fundamentally correct, but if the magnitude or the velocity of the move is insufficient, the position will still incur a loss due to the friction of the premium paid.¹

Therefore, a profitable Nifty 50 options buying algorithm cannot employ standard indicators designed for equity trading, such as simple moving average crossovers or slow oscillators. It must be engineered specifically to detect sudden volatility expansions, micro-structural opening range breakouts, and pure price velocity.

Algorithmic ATM Strike Selection Mathematics

Before an entry signal is generated, the algorithm must dynamically and accurately select the optimal strike price. A pervasive psychological trap for retail algorithmic developers is the inclination to buy deep Out-of-The-Money (OTM) options.²⁵ While the premiums appear mathematically attractive (e.g., a ₹20 premium), the delta of these options is excessively low,

requiring massive, highly improbable market movements (often 5x the premium value) merely to reach profitability within a compressed timeframe.²⁵ Conversely, deep In-The-Money (ITM) options consume excessive capital and frequently suffer from wider bid-ask spreads, exacerbating slippage on market orders. The mathematical consensus for intraday option buying algorithms is to strictly target At-The-Money (ATM) or slightly ITM options, maximizing delta responsiveness while controlling premium outlay.²⁵

To achieve this programmatically in Python or C#, the algorithm must retrieve the live spot price of the Nifty 50 index via the WebSocket feed and utilize a modulo operation against the index's standard exchange-mandated strike interval, which is fixed at 50 for the Nifty index.²⁶

The logic calculates the remainder of the live spot price divided by 50. If the calculated remainder is greater than or equal to 25, the algorithm mathematically rounds up to the next strike interval. If the remainder is less than 25, it rounds down to the nearest interval.²⁶

The programmatic representation follows this formulation:

$$\text{Remainder} = \text{Spot_Price} \pmod{50}$$

$$\text{If Remainder} \geq 25, \text{ATM} = \text{Spot_Price} - \text{Remainder} + 50$$

$$\text{If Remainder} < 25, \text{ATM} = \text{Spot_Price} - \text{Remainder}$$

Execution Example: If the live Nifty spot price ticks at 22,430.

$$22430 \pmod{50} = 30$$

Because 30 is strictly greater than 25, the algorithm identifies the nearest interval and adds the differential, resulting in a precise ATM strike target of 22,450. The system immediately queries the previously parsed DataFrame to extract the specific SECURITY_ID for both the 22,450 Call (CE) and Put (PE) contracts for the current active weekly expiry, holding these tokens in active memory for instant execution.²²

The "Eagle Nifty T315" Breakout Protocol

One of the most robust mechanical frameworks for capturing severe intraday momentum is the opening range breakout. Institutional order flow, algorithmic block trades, and overnight gap resolutions are heavily concentrated in the first hour of the Indian trading session, frequently establishing the day's dominant directional trend. The "Eagle Nifty T315" strategy exploits this specific market microstructure by isolating the initial volatility.²⁸

Forward testing and live execution of this specific logic on Nifty options data have demonstrated an exceptional statistical win rate exceeding 81.8%, yielding consistent positive

mathematical expectancy provided that strict trailing risk controls are applied.²⁸

The algorithmic execution flow for the Eagle Nifty T315 protocol is rigidly defined as follows:

1. **Time-Bound Range Definition:** The system monitors and records the absolute maximum High and minimum Low of the Nifty 50 spot index strictly between the timestamps of 09:15 AM and 09:30 AM, constituting the first 15-minute macroeconomic candle.²⁸
2. **Breakout Trigger Mechanism:** Post 09:30 AM, the algorithm continuously tracks the Last Traded Price (LTP) at a millisecond resolution. If the LTP definitively breaches the recorded 15-minute High, a Call (CE) buying condition is immediately triggered. Conversely, if the LTP violently breaches the 15-minute Low, a Put (PE) buying condition is triggered, assuming a downward trend.²⁹
3. **Retracement and Validation:** Advanced algorithmic variations of the T315 logic do not buy blindly on the exact micro-tick of the breakout, as this invites severe slippage and exposure to false algorithmic sweeps. To mitigate false breakouts (bull/bear traps), the algorithm monitors live retracement levels, ensuring the breakout maintains structural support and volume confirmation before transmitting the POST /order payload.²⁸
4. **Trend Dependency and Volatility Filtering:** This specific breakout strategy is exclusively designed for highly trending days. On range-bound, low-volume, or highly choppy days, minor price reversals will trigger repeated false entries, leading to rapid, systemic capital erosion.²⁸ Consequently, the algorithm requires a secondary quantitative volatility filter. The system must verify that the Average True Range (ATR) or the India VIX is above a specific baseline threshold (e.g., $VIX > 14$) prior to authorizing entry.²⁵ Furthermore, options buying when the VIX is exceptionally high (e.g., > 18) results in purchasing highly inflated premiums, which will suffer devastating "IV crush" even if the directional bet is correct; therefore, a maximum volatility cap must also be enforced.²⁵

The 5 EMA High-Frequency Scalping Strategy

For market environments where the opening range is not cleanly broken or the macro trend is established later in the session, the algorithm must dynamically pivot to high-frequency momentum scalping. The 5-period Exponential Moving Average (EMA) strategy, heavily popularized in the Indian derivatives markets by quantitative traders and educators such as Subasish Pani ("Power of Stocks"), is mathematically elegant and highly effective for algorithmic adaptation.³⁰

The 5 EMA strategy relies on identifying extreme, unsustainable short-term deviations from the mean in a trending environment, betting on a rapid snap-back or aggressive trend continuation. Because it requires absolute precision, it is virtually impossible to trade manually with consistency, making it a perfect candidate for sub-20ms algorithmic execution.

The algorithmic rules for a short-selling equivalent (buying Puts on downward momentum) based on the 5 EMA are strictly defined:

1. **Indicator Initialization:** An Exponential Moving Average with a length parameter of

exactly 5 is calculated and plotted dynamically against a 5-minute or 15-minute Nifty 50 spot chart array.³¹

2. **Alert Candle Detection:** The algorithm continuously scans the array for an "alert candle." An alert candle is defined mathematically as a specific time period where the entire price range of the candle—including its absolute low—is completely detached and trading entirely above the 5 EMA line. This signals severe short-term overextension and an impending reversion.³¹
3. **Trigger Execution:** The algorithm locks onto the low price of this alert candle. The exact millisecond the Last Traded Price (LTP) of any subsequent candle breaks below this specific low, the algorithm fires an aggressive MARKET order payload to buy an ATM Put (PE) option.³¹
4. **Rigid Stop-Loss Placement:** The strict stop-loss parameter is calculated and placed precisely at the absolute high of the alert candle.³¹ Because the entire premise of the setup relies on immediate momentum reversal, any failure of the price to drop instantly invalidates the mathematical premise, necessitating a rigid, mechanical exit without hesitation or modification.

Dynamic Exit Logic and Theta Decay Management

In automated option selling systems, the algorithm can afford to be patient, as the passage of time generates positive yield through the collection of extrinsic value. In an option buying system, patience is a severe mathematical liability. The Greek variable Theta (\$ \Theta \$) represents the daily, quantifiable erosion of an option's extrinsic value.³⁴

Theta decay is not a linear function; it operates on an aggressively steepening curve.³⁴ For At-The-Money (ATM) options—which inherently contain the highest concentration of extrinsic uncertainty—decay accelerates exponentially in the final days and hours leading up to the Thursday weekly expiration.³⁵ A Nifty 50 position held during a period of lateral market consolidation will lose value continuously, perfectly simulating the effect of a "melting ice cream cone".³⁴ Therefore, the exit logic of the options buying algorithm is arguably more critical to long-term profitability than its entry logic. An optimal entry without a ruthless exit protocol guarantees negative expectancy over a large sample size.

Multi-Tiered Dynamic Stop-Loss and Trailing Mechanisms

To protect the trading capital from both sudden, violent market reversals and the silent, continuous erosion of theta decay, the algorithm must implement a dynamic, multi-tiered exit protocol rather than relying on static, arbitrary percentages. The forward-tested data of the Eagle Nifty T315 framework highlights the absolute necessity of dynamic trailing logic to lock in profits during trending phases while simultaneously minimizing exposure during choppy intraday reversals.²⁸

1. The Volatility-Adjusted Initial Stop-Loss: Retail traders frequently utilize a fixed point-based stop-loss (e.g., a ₹20 absolute drop on the option premium). This is mathematically

flawed, as it ignores the real-time volatility of the underlying index. The initial algorithmic stop-loss must be dynamically calculated based on the underlying instrument's real-time volatility metrics.²⁸ Utilizing the Average True Range (ATR), the system calculates a stop-loss that sits mathematically just outside the threshold of normal, acceptable market noise (e.g., $1.5 \times \text{ATR}$).³⁷ This dynamic adjustment prevents premature system stops during standard intraday fluctuations, a common cause of "whipsaw" losses.³⁸

2. Time-Based Partial Exits (Theta Mitigation): Because options inherently lose value every single minute the underlying asset fails to exhibit directional movement, the algorithm must incorporate a strict time-decay failsafe.²⁸ If the position has been open for a specified duration (e.g., 45 minutes) and the target profit percentage has not been reached, the algorithm must execute a forced liquidation or at minimum, a 50% partial exit.²⁸ This time-based exit acknowledges that the momentum premise of the initial trade trigger has failed. Holding the position further merely subsidizes the option seller's theta collection and exposes the algorithm to negative gamma.³⁶ For positional carrying, the algorithm must enforce a strict rule to exit all underwater positions by Wednesday 2 PM, as Thursday's zero-day-to-expiry (ODTE) theta decay is mathematically brutal and unrecoverable.²⁵

3. Dynamic Trailing Stop Logic: Once a trade moves definitively into profitability, securing those mathematical gains is paramount. The algorithm must implement a step-trailing mechanism.³⁸ For instance, upon reaching an initial 20% profit threshold on the premium, the absolute stop-loss is automatically amended via the INDstocks /order modification endpoint to the exact break-even execution price. For every subsequent 10% increase in premium, the trailing stop is raised by a corresponding 10%. Advanced indicators such as the Parabolic SAR (Stop and Reverse) or Chandelier Exits can be programmed directly into the system's execution array to automate this trailing based on pure price action rather than arbitrary, hardcoded percentages.³⁸

Failsafe Architecture and Systemic Risk Management

An algorithmic system operating directly on live financial APIs is exposed to severe, potentially catastrophic operational and market risks. An unhandled syntax exception, an infinite while loop in the order placement function, or a sudden loss of WebSocket connectivity can result in rapid capital destruction within seconds.⁴¹ Institutional architecture must treat risk management not merely as a supplementary feature, but as a core parallel subsystem operating independently of the primary execution logic.⁴²

Capital Allocation and Position Sizing Limits

The cardinal rule of derivatives trading, heavily emphasized by experts, is capital preservation.²⁵ Regardless of the algorithm's historical win rate or the perceived perfection of the setup, maximum drawdown limits must be strictly hardcoded into the execution logic before any payload is transmitted.²⁸

The system should enforce a rigid, mathematical position sizing formula. For a standard Nifty options buying strategy, the absolute maximum capital risked per trade should never exceed 2% to 3% of the total account equity.²⁵ If the total allocated trading capital is ₹50,000 (the recommended minimum for meaningful options trading to maintain a 30-40% buffer), the absolute maximum permissible loss per trade is capped at ₹1,500.²⁵

The algorithm calculates the required lot size dynamically prior to order placement using the following formulation:

$$\text{Max Lots} = \frac{\text{Capital Risk Limit}}{\text{Stop Loss Value per Share} \times \text{Nifty Lot Size}}$$

If the calculated lot size is less than 1, the algorithm rejects the trade entirely, logging a risk violation. Furthermore, the system must enforce strict, unyielding rules against "revenge trading" or averaging down. The algorithm must never increase position size or double lot allocations following a string of losses; position scaling should only occur incrementally and systematically after a sustained sequence of profitable executions.²⁵

Circuit Breakers, Daily Limits, and The "Kill Switch"

To protect against anomalous black-swan market events, flash crashes, or logic malfunctions within the Gemini LLM agent, the infrastructure requires external boundaries that sit above all individual trading strategies.³⁷

1. Daily Drawdown Circuit Breakers: A global variable within the local SQLite database must track the cumulative intraday Profit and Loss (PnL) across all closed and open positions. If the system hits a predefined daily drawdown limit (e.g., -5% of total capital), a software circuit breaker is immediately triggered.³⁷ The algorithm's risk manager module instantly executes SELL market orders for all open derivatives, cancels all pending LIMIT orders via the INDstocks /order API, and actively suspends all further trading activity and WebSocket polling until the initialization of the following trading session.⁴¹

2. The Hard Kill Switch: A manual intervention protocol is both a regulatory requirement and a practical necessity for automated systems.⁴³ The Google Antigravity IDE environment allows for a dedicated Agent Skill to act as a systemic Kill Switch.¹² Should the human developer or the Agent Manager detect an infrastructure failure, an API desync, or a massive macroeconomic shock, invoking the kill switch bypasses all strategic logic. It interfaces directly with the INDstocks broker API to violently liquidate the portfolio at the current market price and gracefully terminates the WebSocket connections, isolating the account from any further exposure.³⁷

3. API Rate Limit Safeguards and Throttling: Given INDstocks' strict limit of 10 orders per second and a maximum of 25 modifications per specific order, aggressive, high-frequency strategies like the 5 EMA scalping must be meticulously throttled.²¹ The order execution module must queue outbound HTTP requests and monitor the exact timestamps of recent API calls. If

the velocity of orders approaches 8 per second, the algorithm must forcefully pause execution for 1000 milliseconds to prevent an HTTP 429 "Too Many Requests" ban from the broker's servers.²³ A ban would leave open positions completely unmanaged, rendering stop-losses ineffective and exposing the portfolio to infinite risk.

Product Requirements Document (PRD)

To synthesize the strategy, infrastructure, and execution mechanics into an actionable, enterprise-grade development roadmap, the following Product Requirements Document details the exact quantitative specifications required to build the Nifty 50 Options Algorithmic System using the Google Antigravity IDE and the INDstocks API.

1. System Architecture and Environment Deployment

System Component	Technical Specification & Setup	Operational Purpose & Integration
Development IDE	Google Antigravity (Agent-Assisted Mode)	To orchestrate autonomous coding agents, deploy self-healing loops for syntax errors, and utilize visual browser verification for UI dashboards. ³
Reasoning Engine	Gemini 3 Pro via Agent Manager	Acts as the primary architectural brain for translating complex quantitative logic and PR Sundar's methodologies into robust Python execution syntax. ⁹
Context Protocol	Model Context Protocol (MCP) Python SDK	Securely integrates INDstocks API documentation and handles Authorization header token passing without exposing hardcoded keys in the repository. ¹³
Execution Modularity	Antigravity Workspace Skills	Encapsulates specific trading capabilities (e.g.,

	(.agent/skills/)	calculate_theta, execute_indstocks_order) to prevent LLM context saturation and reduce token costs. ¹¹
Validation Logging	AgentKit with JSON Lines (.jsonl)	Wraps every algorithmic task in a validation loop (max 3 retries). Logs errors and execution times to allow the developer to identify recurring agentic failure patterns. ¹⁰

2. Market Data Pipeline and API Integrations

Data Requirement	INDstocks Endpoint / Mechanism	Algorithmic Processing Logic
Instrument Master	GET /market/instruments?source=fno	Downloads the daily raw CSV. Algorithm parses and filters explicitly for NIFTY, CE/PE, nearest expiry, and targeted strike to extract the critical SECURITY_ID. ²²
Live Price Feed	wss://ws-prices.indstocks.com/.../prices	Subscribes using JSON payload "action":"subscribe" and "mode":"ltp". Maps the SECURITY_ID with the mandatory NFO: segment prefix. ¹⁷
Order Execution	POST /order	Pushes authenticated JSON payload containing txn_type, segment: "DERIVATIVE", algo_id: "99999", and lot-compliant integer quantities. ¹⁷
Trade Status Updates	wss://ws-order-updates.ind	Subscribes to

	stocks.com...	"mode":"order_updates" to capture asynchronous fills, rejections, and precise execution prices without relying on rate-heavy REST API polling. ¹⁷
Rate Limit Handler	Local Request Queue & Token Bucket	Enforces a hard cap of 8 orders per second (below the 10/sec limit) and 90 data requests per second (below the 100/sec limit) to entirely prevent HTTP 429 errors. ²³

3. Quantitative Trading Logic (The Buy Engine)

Logic Module	Strategy Parameters & Thresholds	Execution Trigger Mechanics
Strike Selection	Dynamic ATM calculation algorithm	Spot_Price % 50. Rounds up if remainder $\geq$, otherwise rounds down. Extracts and stores the corresponding Call or Put contract Security ID. ²⁶
Volatility Filter	India VIX / ATR Baseline Thresholds	Strategy execution is completely suspended if India VIX is below 14 (premiums too expensive relative to movement) or above 18 (massive IV crush risk). ²⁵
Primary Strategy	Eagle Nifty T315 Breakout Framework	Maps the absolute High and Low of the 09:15 to 09:30 AM candle. Executes market order strictly upon breakout of this range with confirmed retracement

		support. ²⁸
Secondary Strategy	5 EMA Momentum Scalping (Power of Stocks)	Identifies a 5-minute candle fully detached above the 5 EMA. Executes an aggressive PE order the exact millisecond the low of this alert candle is breached. ³¹

4. Dynamic Exit and Systemic Risk Management Protocols

Risk Protocol	Mathematical Trigger & Calculation	Autonomous System Action
Volatility Stop-Loss	$1.5 \times$ 5-minute Average True Range (ATR)	Sets the initial absolute floor beneath the entry price to absorb normal market noise, preventing "whipsaw" exits common with fixed-point stops. ³⁷
Theta Mitigation	Time-in-Trade > 45 minutes	If target profit is unmet after 45 minutes of sideways consolidation, the system forcefully closes the position to prevent accelerating extrinsic value decay. ²⁸
Trailing Profits	20% Initial Profit premium threshold	Upgrades stop-loss immediately to break-even. Implements Chandelier Exit or step-trailing logic to lock in massive gains dynamically during directional momentum. ³⁸
Capital Protection	2% Account Equity Limit per Trade	Dynamically sizes lots before execution. Rejects any trade signal where the

		required minimum lot size exceeds the 2% maximum allowable loss threshold. ²⁵
System Failsafe	-5% Daily Drawdown OR Manual Override	Global Kill Switch: Instantly fires SELL MARKET for all open derivatives, purges all pending modifications, and terminates WebSocket data feeds. ³⁷

The successful deployment of this sophisticated architecture transitions the highly volatile act of Nifty 50 options buying from a discretionary, emotion-driven gamble into a precise, mathematically rigorous quantitative discipline. By leaning heavily on the autonomous agentic capabilities of the Google Antigravity IDE and the Model Context Protocol to manage the boilerplate infrastructural complexity, the developer is freed to focus exclusively on refining the mathematical edges, optimizing execution latency, and ensuring the absolute, uncompromising integrity of the system's risk boundaries.

Works cited

1. OPTIONS BUYING vs SELLING | Mission Options E13 - YouTube, accessed April 30, 2026, <https://www.youtube.com/watch?v=R8bsVwmVX40>
2. Trading income is one part of my business: PR Sundar - Mint, accessed April 30, 2026, <https://www.livemint.com/money/personal-finance/people-lose-when-options-are-used-for-speculation-11668704923600.html>
3. Tutorial : Getting Started with Google Antigravity | by Romin Irani - Medium, accessed April 30, 2026, <https://medium.com/google-cloud/tutorial-getting-started-with-google-antigravity-b5cc74c103c2>
4. Build with Google Antigravity, our new agentic development platform, accessed April 30, 2026, <https://developers.googleblog.com/build-with-google-antigravity-our-new-agentic-development-platform/>
5. Introducing Google Antigravity, a New Era in AI-Assisted Software Development, accessed April 30, 2026, <https://antigravity.google/blog/introducing-google-antigravity>
6. Google Antigravity Full Tutorial for Beginners - YouTube, accessed April 30, 2026, <https://www.youtube.com/watch?v=EtuWP9Ugo34&vl=en-US>
7. Google Antigravity Hands on : Explained Visually and step by step demo - YouTube, accessed April 30, 2026, <https://www.youtube.com/watch?v=8aNv221Vvu8>
8. Getting Started with Google Antigravity - Codelabs, accessed April 30, 2026,

- <https://codelabs.developers.google.com/getting-started-google-antigravity>
9. Google Antigravity: Beginner Guide to the New Agentic IDE (Step-by-Step + Real Use Case), accessed April 30, 2026,
<https://medium.com/google-developer-experts/google-antigravity-beginner-guide-to-the-new-agentic-ide-step-by-step-real-use-case-9585578e7308>
 10. Building Validation Loops in Antigravity Agents: 3 Patterns for AI-Verified Output, accessed April 30, 2026,
<https://antigravitylab.net/en/articles/agents/antigravity-agent-validation-loop-three-patterns>
 11. Authoring Google Antigravity Skills - Codelabs, accessed April 30, 2026,
<https://codelabs.developers.google.com/getting-started-with-antigravity-skills>
 12. Tutorial : Getting Started with Google Antigravity Skills, accessed April 30, 2026,
<https://medium.com/google-cloud/tutorial-getting-started-with-antigravity-skills-864041811e0d>
 13. modelcontextprotocol/python-sdk: The official Python SDK for Model Context Protocol servers and clients - GitHub, accessed April 30, 2026,
<https://github.com/modelcontextprotocol/python-sdk>
 14. Build an MCP server - Model Context Protocol, accessed April 30, 2026,
<https://modelcontextprotocol.io/docs/develop/build-server>
 15. Google Antigravity Complete Analysis: How the Agent-First IDE is Reshaping the Future of Software Development, accessed April 30, 2026,
https://www.youngju.dev/blog/llm/antigravity_ai_analysis.en
 16. Antigravity Editor: MCP Integration, accessed April 30, 2026,
<https://antigravity.google/docs/mcp>
 17. WebSocket API - Real-time Market Data Streaming - INDstocks API ..., accessed April 30, 2026, <https://api-docs.indstocks.com/Websockets/>
 18. Antigravity + Claude Opus 4.6 = Incredible : r/google_antigravity - Reddit, accessed April 30, 2026,
https://www.reddit.com/r/google_antigravity/comments/1r74ikf/antigravity_claude_opus_46_incredible/
 19. API-TRADING - INDstocks, accessed April 30, 2026,
<https://www.indstocks.com/features/api-trading>
 20. Complete API Overview - INDstocks Trading API Catalog, accessed April 30, 2026, <https://api-docs.indstocks.com/api-overview/>
 21. Frequently Asked Questions (FAQ) - INDstocks Trading API, accessed April 30, 2026, <https://api-docs.indstocks.com/faq/>
 22. Getting Started - INDstocks Trading API Tutorial, accessed April 30, 2026,
<https://api-docs.indstocks.com/getting-started/>
 23. API Conventions - INDstocks Trading API Standards, accessed April 30, 2026,
<https://api-docs.indstocks.com/conventions/>
 24. Option_SELLING_Webinar_0321.pdf - Slideshare, accessed April 30, 2026,
<https://www.slideshare.net/slideshow/optionsellingwebinar0321pdf/255110294>
 25. Nifty Options Strategies for Beginners: 5 Simple Setups - PL Capital, accessed April 30, 2026,
<https://www.plindia.com/blogs/nifty-options-strategies-for-beginners/>

26. How to find ATM for NIFTY - Kite Connect developer forum, accessed April 30, 2026, <https://kite.trade/forum/discussion/9887/how-to-find-atm-for-nifty>
27. Algorithm to Calculate AT THE MONEY (ATM) Strike price Using C# | by kamal chanchal, accessed April 30, 2026, <https://medium.com/@kchanchal78/algorithm-to-calculate-at-the-money-atm-strike-price-using-c-9d8841c405a2>
28. Nifty Strategy: 81% Wins & ₹33K Profit — Thoughts on Exit Logic? : r/algotrading - Reddit, accessed April 30, 2026, https://www.reddit.com/r/algotrading/comments/1mqk7gx/nifty_strategy_81_wins_33k_profit_thoughts_on/
29. Building a Robust Trading Strategy on Nifty50 Stock Options: A Step-by-Step Guide, accessed April 30, 2026, <https://tradetron.tech/blog/building-a-robust-trading-strategy-on-nifty50-stock-options-a-step-by-step-guide>
30. Inside Candle - AlgoTest Documentation, accessed April 30, 2026, <https://docs.algotest.in/signals/famous-strategies/inside-candle/>
31. 5 EMA Scalping Strategy - AlgoTest Documentation, accessed April 30, 2026, <https://docs.algotest.in/signals/famous-strategies/5-ema/>
32. Best Trading Strategy by Power of Stocks | Learn Trading | Subasish Pani Traffic Light Setup, accessed April 30, 2026, <https://www.youtube.com/watch?v=mMqXcoH1H3k>
33. Automate 5 EMA strategy of @POWEROFSTOCKSBySubasish on AlgoTest | Algo Trading, accessed April 30, 2026, <https://www.youtube.com/watch?v=gk7H8xflydo>
34. Theta Decay Explained: The Silent Killer of Options Buyers (Options Trading for Beginners) - YouTube, accessed April 30, 2026, <https://www.youtube.com/watch?v=6rCHSB6GJyA>
35. Understanding Theta and Time Decay in Options | IBKR Quant, accessed April 30, 2026, <https://www.interactivebrokers.com/campus/ibkr-quant-news/understanding-the-ta-and-time-decay-in-options/>
36. Theta Decay in Options Trading: 3 Strategies | Charles Schwab, accessed April 30, 2026, <https://www.schwab.com/learn/story/theta-decay-options-trading>
37. How to Set Up Algorithmic Trading: Complete Beginner-to-Pro Setup Guide (2026), accessed April 30, 2026, <https://www.quantvps.com/blog/how-to-set-up-algorithmic-trading>
38. exit strategies : r/algotrading - Reddit, accessed April 30, 2026, https://www.reddit.com/r/algotrading/comments/1lkca84/exit_strategies/
39. Options lose value every day: Measure it with Theta decay - PyQuant News, accessed April 30, 2026, <https://www.pyquantnews.com/the-pyquant-newsletter/options-lose-value-every-day-measure-theta-decay>
40. Glossary - Ventura, accessed April 30, 2026, <https://www.venturasecurities.com/share-market-glossary/glossary/>
41. Risk Management Strategies for Algo Trading - LuxAlgo, accessed April 30, 2026,

- <https://www.luxalgo.com/blog/risk-management-strategies-for-algo-trading/>
42. All algos are not equal - ION Group, accessed April 30, 2026,
<https://iongroup.com/blog/markets/all-algos-are-not-equal/>
 43. Algorithmic Trading Under MiFID II: Increased Regulatory Expectations and Annual Self-assessment - Kroll, accessed April 30, 2026,
<https://www.kroll.com/en/publications/financial-compliance-regulation/algorithmic-trading-under-mifid-ii>
 44. Options Trading Workshops, accessed April 30, 2026,
<https://workshop.prsundar.com/>
 45. Best Practices For Automated Trading Risk Controls And System Safeguards - FIA.org, accessed April 30, 2026,
https://www.fia.org/sites/default/files/2024-07/FIA_WP_AUTOMATED%20TRADING%20RISK%20CONTROLS_FINAL_0.pdf
 46. Algo trading research with Google's Antigravity and Gemini 3 - YouTube, accessed April 30, 2026, <https://www.youtube.com/watch?v=ypFG006G4WQ>